1.1 Introduction to Project Theme:

In modern software development, managing changes to source code efficiently has become a critical requirement. As software applications increase in size and complexity, multiple developers often work simultaneously, making frequent updates, enhancements, and bug fixes. Without a structured mechanism to track these changes, maintaining consistency, reliability, and accuracy of the code becomes difficult. This challenge is effectively addressed by **Version Control Systems (VCS)**, which maintain a record of file modifications and preserve historical versions of data.

The proposed project focuses on the development of a **simplified web-based Version Control System**, inspired by platforms such as GitHub. The objective is not to replicate the complete distributed architecture of Git, but to demonstrate the **core principles of version management** in a simplified and understandable manner. The system primarily concentrates on repository creation, file versioning, and commit history tracking, thereby reducing technical complexity while preserving the essential functionality of a version control system.

This project is designed as an **academic prototype** using modern web technologies. It provides users with a centralized platform to manage repositories and monitor file changes over time. By implementing this system, the project aims to bridge the gap between theoretical concepts of version control and their practical application in real-world software development environments.

The core objective of the project is to replace traditional manual processes with a structured, database system that stores all information in tabular format using a SQL database. This structured approach allows quick access to data, better monitoring and accurate maintenance of records. The system also supports easy generation of reports, which helps to analyse data, track performance and make informed decisions.

Additionally, the system emphasizes usability and accessibility by providing an intuitive web interface that allows users to perform version control operations without requiring advanced technical knowledge of command-line tools. Features such as user authentication, repository dashboards, commit logs, and file tracking mechanisms are integrated to ensure a smooth and organized workflow. Security measures are also considered to protect repository data and maintain user privacy. By combining simplicity with essential version control functionalities, the project serves as a practical learning tool for students and beginners, enabling them to understand collaborative development processes.

1.2 Working of System:

The Repository Management System follows a client–server architecture, where the frontend communicates with the backend using HTTP requests and responses. Users interact with the system through a web-based interface developed using HTML, CSS, and JavaScript, which ensures ease of access and usability.

The working of the system can be summarized as follows:

- Users access the system and perform authentication through a secure login process.
- After successful authentication, users are redirected to a structured dashboard.
- Users can create new repositories, upload files, and modify existing files through the dashboard interface.

Each user request is validated and processed by the backend developed using **Python and the Flask framework.**

Whenever a user modifies a file and saves the changes, the system creates a new version entry instead of overwriting the existing file. Each version contains:

- Version number
- Updated file content
- Commit message
- Timestamp
- Author information

All version-related data is stored securely in a MySQL database, allowing users to view, compare, or restore previous versions when required. This process effectively simulates the concept of commits found in traditional version control systems.

Furthermore, the system maintains a detailed commit history for every repository, enabling users to track the sequence of modifications made over time. Each operation performed by the user is logged to ensure transparency and accountability in the development process. The centralized database architecture ensures data consistency and prevents loss of information during concurrent operations. By automating version tracking and repository management, the system reduces manual effort, minimizes errors, and improves overall workflow efficiency. This structured working mechanism makes the platform reliable for academic demonstration and practical understanding of version control principles.

1.3 Need & Scope of Proposed System:

Need of the Proposed System -

The need for the proposed system arises from the growing importance of version control knowledge in the software development industry. Although professional platforms like GitHub provide advanced features, their internal working mechanisms are often complex and difficult for beginners to understand.

The proposed system addresses the following needs:

- Simplified understanding of version control concepts
- Centralized repository management
- Elimination of manual file backups
- Prevention of accidental data loss
- Improved tracking of file changes and updates

By maintaining a structured version history, the system improves accountability and enables users to revert to earlier versions whenever required.

Scope of the Proposed System -

The scope of the system is primarily limited to basic version control functionalities suitable for academic and learning purposes. These include:

- Repository creation and management
- File versioning
- Commit history tracking
- Centralized data storage

Although limited in scope, the system is designed to be scalable. In the future, it can be extended to include:

- Branching and merging
- Role-based access control
- Detailed activity logs
- Advanced version comparison tools

Thus, the system serves as a strong foundation for learning and future enhancements.

2.1 Objectives of the System:

The main objectives of the MiniGit system are:

- To provide a **user-friendly platform** for managing repositories without complex Git commands.
- To allow users to **explore system features before registration** through pre-register pages.
- To enable registered users to **create, manage, and monitor repositories** efficiently.
- To maintain a clear distinction between **pre-register and post-register functionalities**.
- To provide **secure user authentication and authorization**.
- To track user activity such as repository updates, commits, and issues.
- To offer a **centralized dashboard** for quick access to recent repositories and activity.
- To support basic collaboration features like issues and commits.
- To ensure **data integrity and consistency** using a structured database.
- To improve productivity for students and beginners learning version control concepts.
- To maintain a detailed commit history that allows users to review and restore previous versions of files.
- To provide an organized file versioning mechanism that prevents accidental data loss.
- To enable efficient repository navigation through structured categorization and search features.
- To generate activity reports that help users monitor their development progress.
- To implement role-based access control for better security and management of user permissions.
- To provide real-time feedback and notifications for important repository activities.
- To minimize manual errors by automating repository and version management processes.
- To ensure system scalability so that it can handle increasing repositories and users efficiently.
- To design an intuitive and responsive interface accessible across different devices and screen sizes.
- To serve as a practical educational tool that demonstrates the fundamental workflow of modern version control systems.

2.2 Software Requirement Specification:

1. Introduction:

i. Purpose of this Document:

This section defines the software requirements of MiniGit, a web-based repository management system that provides basic version control features through a simple and user-friendly interface.

ii. Scope of this Document:

MiniGit provides a streamlined platform where anyone can explore public repositories, view commit histories, and browse issues without needing an account. This makes it easy for users to discover projects and evaluate code before deciding to contribute. Once users register and authenticate securely, they unlock full repository management capabilities. Authenticated users can create both public and private repositories, initialize them with templates, and manage collaborators. They can also perform commits across branches, track changes through detailed histories, and handle merging. Additionally, the issue tracking system allows users to create, assign, and manage issues with labels, milestones, and comments, facilitating effective project management and team collaboration.

2. General Description:

MiniGit is a web-based repository management system built with Flask and MySQL. It allows users to register, create and manage repositories, track commits, and raise issues — similar to a lightweight GitHub. It includes an admin panel with live reporting on users, repositories, commits, issues, feedback, and repository popularity. The frontend uses a dark cyberpunk-style UI with Chart.js for data visualization, and data is loaded dynamically via a REST API pattern where JavaScript fetches JSON from Flask backend endpoints.

3. Functional Requirements:

i. Public User Requirements

1. The system shall display home, features, demo, privacy policy, and help pages.
2. The system shall allow users to submit feedback.
3. The system shall allow users to register an account.

ii. Registered User Requirements

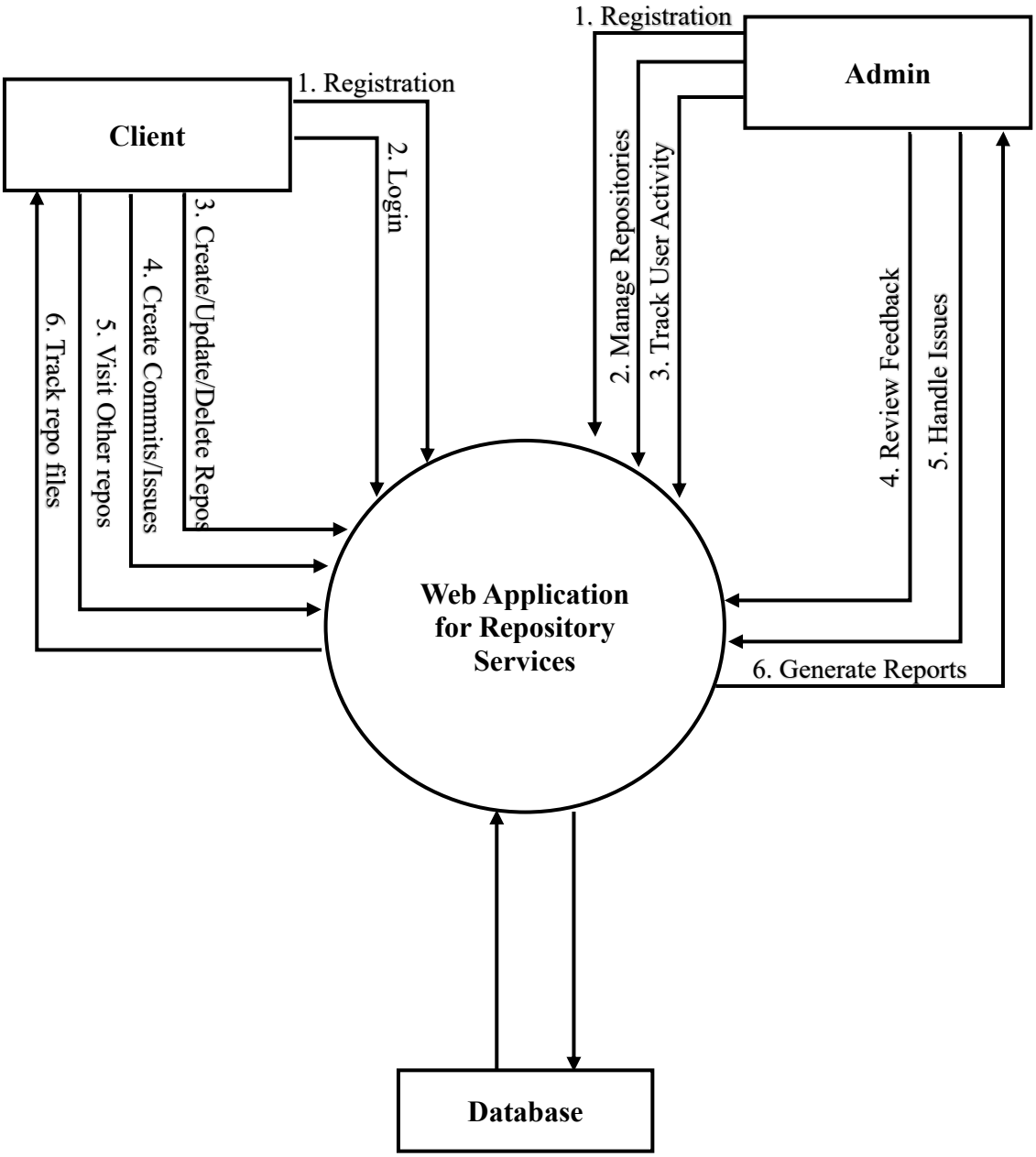
1. The system shall allow secure user login and logout.

2. The system shall display a dashboard with repositories and recent activity.
3. The system shall allow users to create repositories, commits, and issues.
4. The system shall allow users to view repositories and activity history.
5. The system shall allow users to manage their profile

iii. Non-Functional Requirements

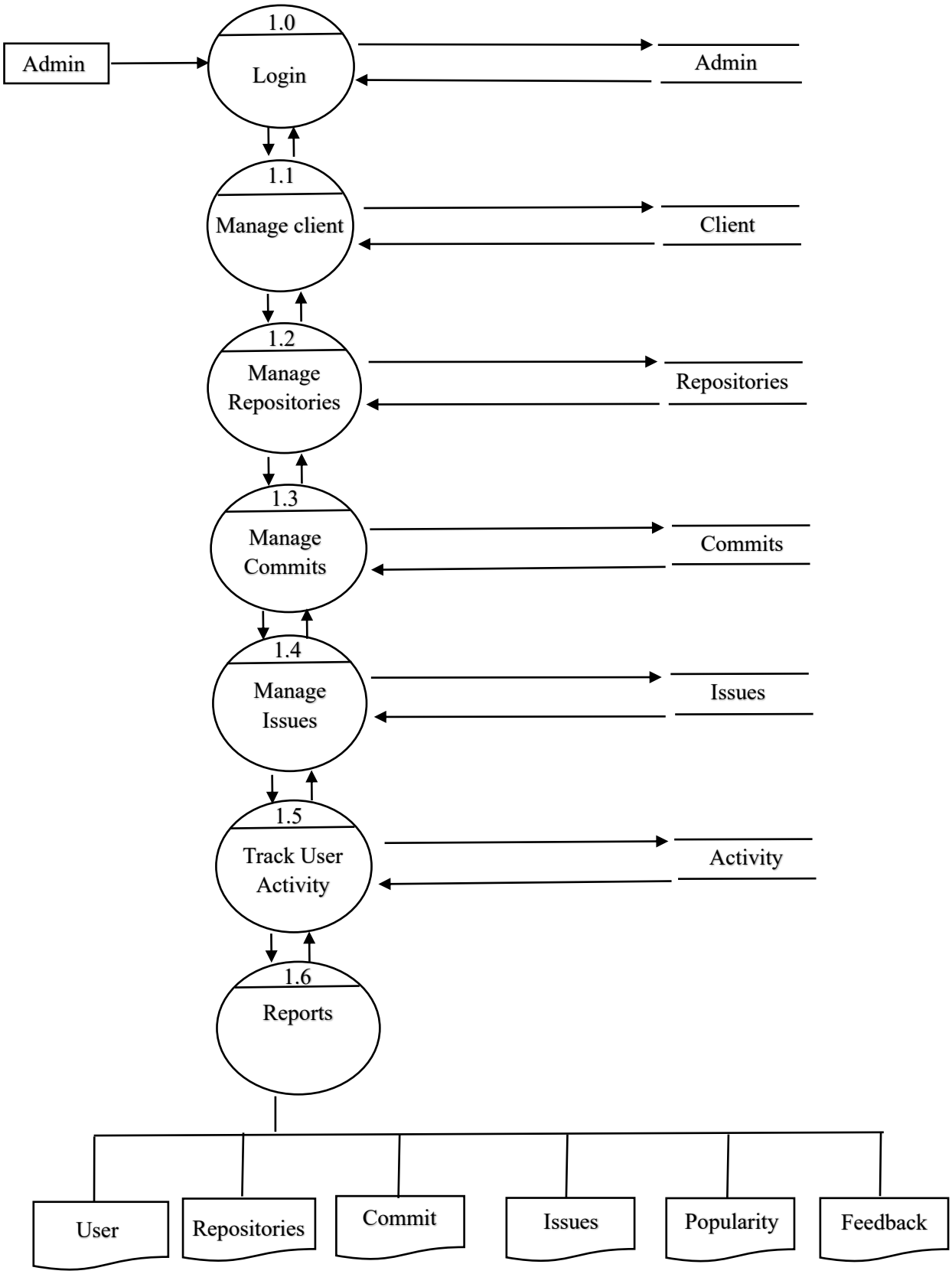
1. The system shall provide fast response time.
2. User data shall be securely stored and encrypted.
3. The system shall be easy to use and reliable.

3.1 Context Level Diagram:

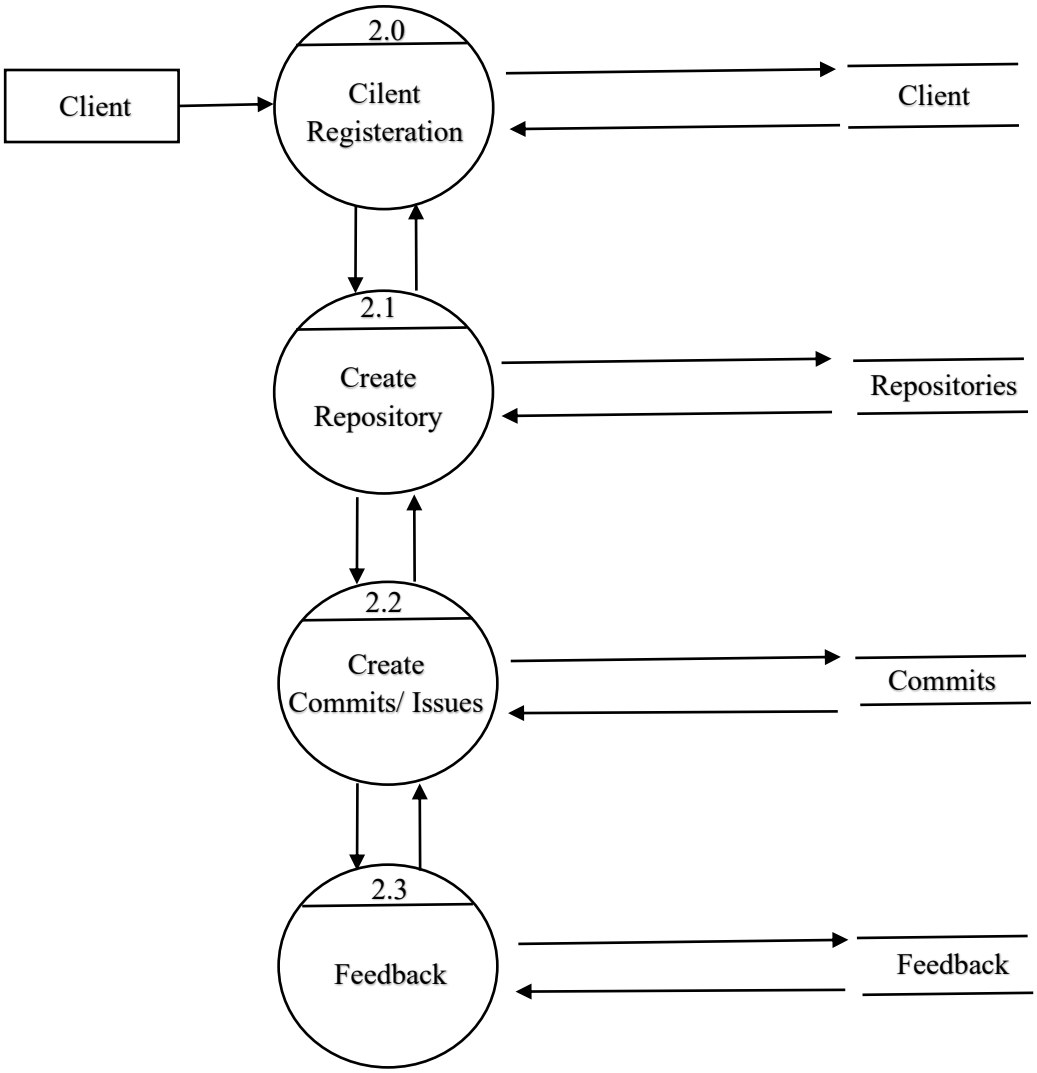


3.2 Data Flow Diagram:

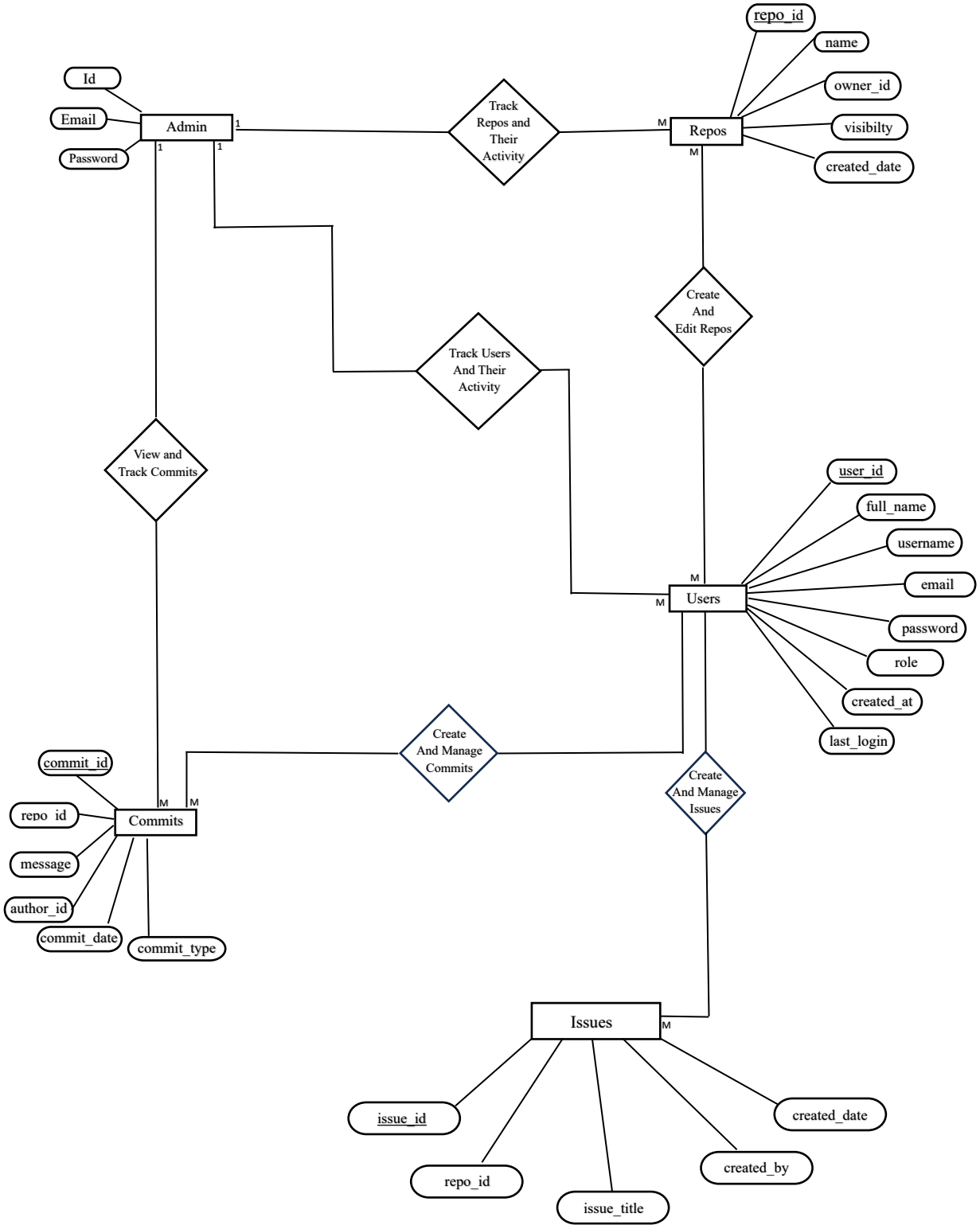
- **First Level DFD for Admin:**



- **First Level Diagram for User:**



3.3 Entity Relationship Diagram:



3.4 System Requirement:

Hardware Requirement:

- Processor: Intel i5 8th Generation / AMD Ryzen 5.
- HDD / SSD: 512 GB.
- RAM: 8 GB.
- System Type: 64-bit operating system, x64-based processor.
- Monitor.
- Keyboard.
- Mouse.

Software Requirement:

- Operating System: Linux or Windows.
- Web Browser: Chrome or Edge.
- Text Editor: Visual Studio Code.
- Database: MySQL.
- Front-end: HTML, CSS, JavaScript.
- Back-end: Python.

4.1 Database Design (Data Dictionary):

i. Table name: Users

Sr. No.	Fields	Data type	Size	Description
1	User_id	Int		To store the user id
2	Full_name	Varchar	100	To store the name
3	Username	Varchar	50	To store the username
4	Email	Varchar	100	To store the email
5	Password	Varchar	255	To store the password
6	Created_at	Timestamp		To store the creation date
7	Last_login	Timestamp		To store the last login

ii. Table name: Repositories

Sr. No.	Fields	Data type	Size	Description
1	Repo_id	Int		To store the repo id
2	Repo_name	Varchar	100	To store the repo name
3	Owner_id	Int		To store the owner id
4	Visibility	Enum		To store the visibility
5	Created_date	Timestamp		To store the created date

iii. Table name: Commits

Sr. No.	Fields	Data type	Size	Description
1	Commit_id	Int		To store the commit id
2	Repo_id	Int		To store the repo id
3	Commit_message	Varchar	500	To store the commit message
4	Author_id	Int		To store the author id
5	Commit_type	Enum		To store the commit type
6	Commit_date	Timestamp		To store the commit date

iv. Table name: Issues

Sr. No.	Fields	Data type	Size	Description
1	Issue_id	Int		To store the issue id

2	Repo_id	Int		To store the repo id
3	Issue_title	Varchar	200	To store the issue title
4	Description	Text		To store the issue description
5	Created_by	Int		To store the creator id
6	Status	Enum		To store the status
7	Priority	Enum		To store the priority
8	Created_date	Timestamp		To store the created date

v. Table name: Repo Views

Sr. No.	Fields	Data type	Size	Description
1	View_id	Int		To store the view id
2	Repo_id	Int		To store the repo id
3	User_id	Int		To store the user id
4	Viewed_at	Timestamp		To store the view time

vi. Table name: Activity Logs

Sr. No.	Fields	Data type	Size	Description
1	Activity_id	Int		To store the activity id
2	User_id	Int		To store the user id
3	Repo_id	Int		To store the repo id
4	Activity_message	Varchar	255	To store the activity message
5	Created_at	Timestamp		To store the creation time

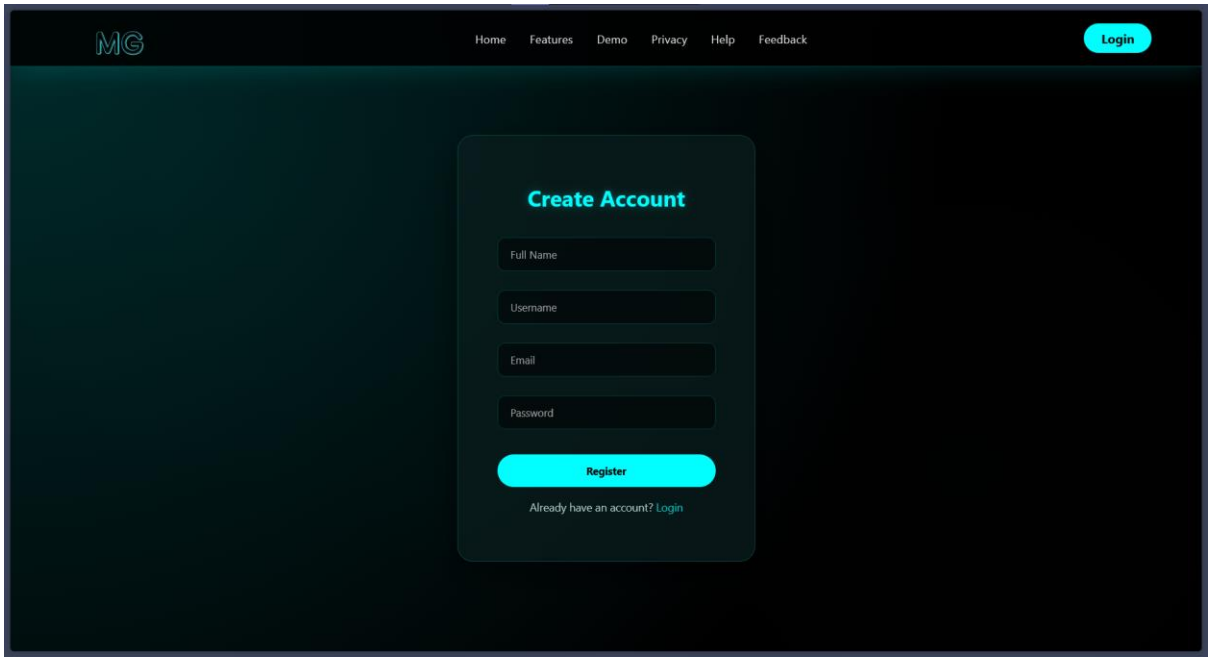
vii. Table name: Feedback

Sr. No.	Fields	Data type	Size	Description
1	Feedback_id	Int		To store the feedback id
2	Name	Varchar	100	To store the name
3	Category	Varchar	50	To store the Category
4	Experience	Varchar	20	To store the experience
5	Message	Text		To store the message
6	Created_at	Timestamp		To store the creation time

4.2 Input Design:

- **Without data:**

register.html:

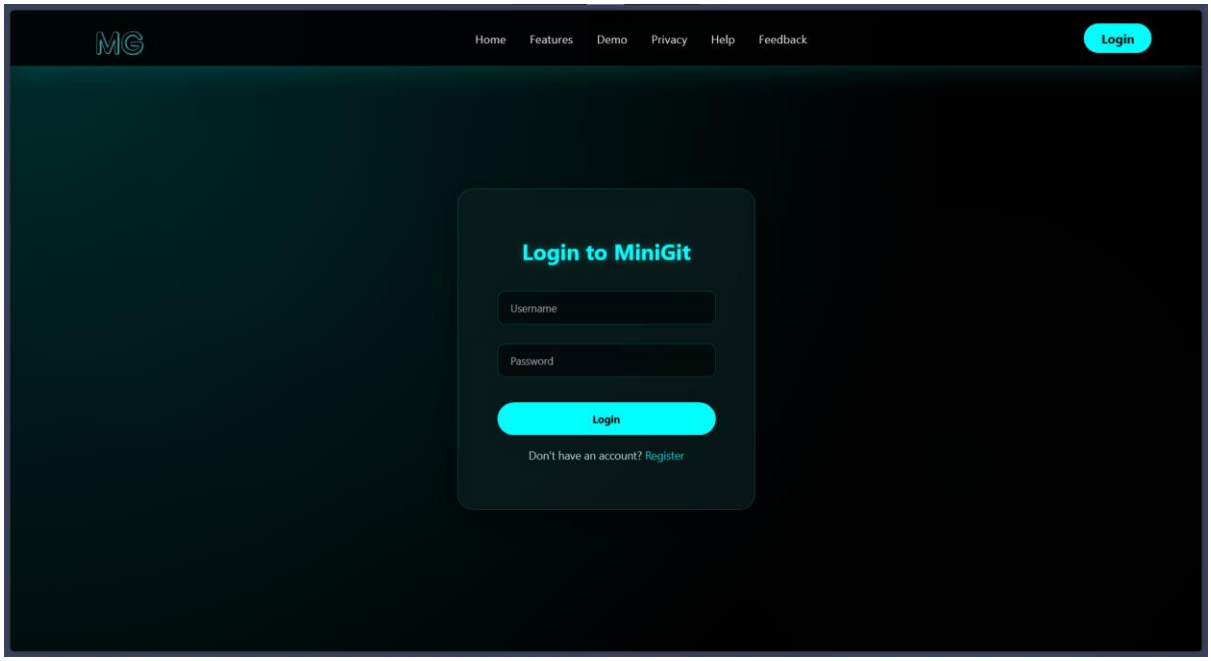


- **Source Code:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Register | MiniGit</title>
  <link    rel="stylesheet"    href="{{    url_for('static',
filename='css/login.css') }}">
</head>
<body>
  <!-- HEADER -->
 <header class="header">
    <div class="brand">
      <img                  src="{{         url_for('static',
filename='/images/blue.png')        }}"        width="150px"
height="150px"></div>
    </div>

    <nav class="nav">
      <a   href="{{   url_for('home')   }}"   class="{%   if
active_page == 'home' %}active{% endif %}">Home</a>
      <a  href="{{  url_for('features')  }}"  class="{%  if
active_page == 'features' %}active{% endif %}">Features</a>
      <a   href="{{   url_for('demo')   }}"   class="{%   if
active_page == 'demo' %}active{% endif %}">Demo</a>
```

```
      <a  href="{{  url_for('privacy')  }}"  class="{%  if
active_page == 'privacy' %}active{% endif %}">Privacy</a>
      <a   href="{{   url_for('help')   }}"   class="{%   if
active_page == 'help' %}active{% endif %}">Help</a>
      <a  href="{{  url_for('feedback')  }}"  class="{%  if
active_page == 'feedback' %}active{% endif %}">Feedback</a>
    </nav>
    <a href="{{ url_for('login') }}" class="login">Login</a>
  </header>
  <!-- REGISTER FORM -->
  <div class="login-container">
    <div class="login-card">
      <h2>Create Account</h2>
      <form method="POST" action="/register">
        <input          type="text"          name="full_name"
placeholder="Full Name" required>
        <input          type="text"          name="username"
placeholder="Username" required>
        <input type="email" name="email" placeholder="Email"
required>
        <input        type="password"        name="password"
placeholder="Password" required>
        <button type="submit">Register</button>
      </form>
      <p>Already    have    an    account?    <a    href="{{
url_for('login')  }}"  style="color:  cyan;  text-decoration:
none;">Login</a>
      </p>
    </div>
  </div>
</body>
</html>
```

login.html:

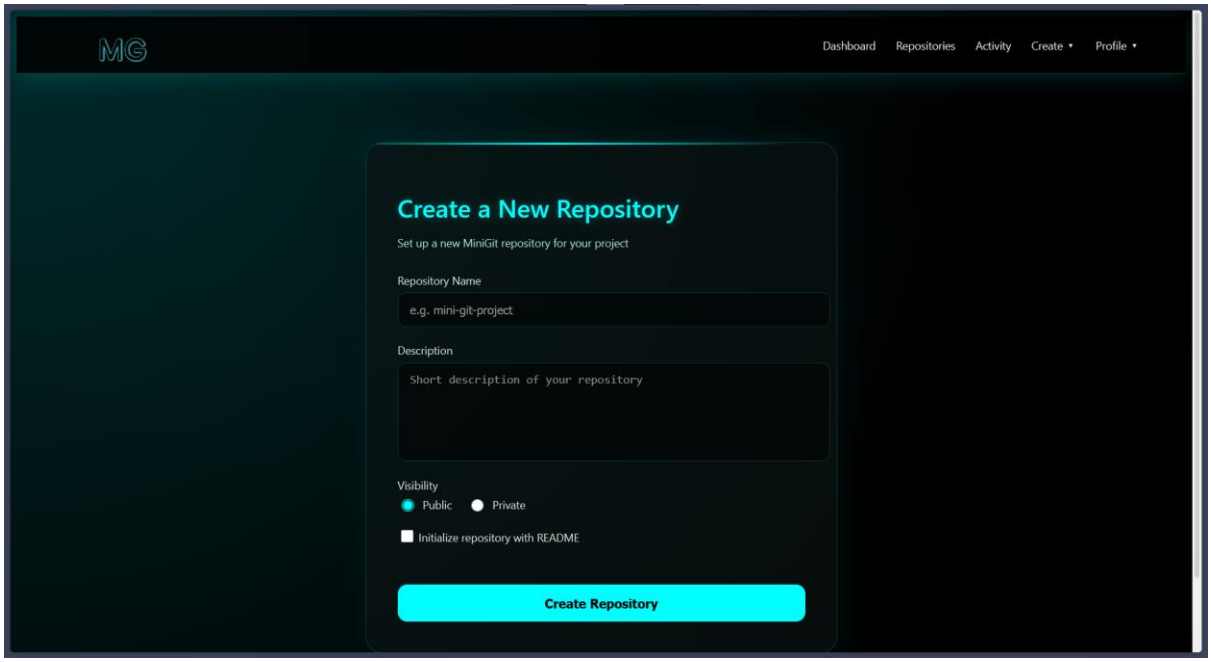


- **Source Code:**

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Login | MiniGit</title>
  <link rel="stylesheet" href="/static/css/login.css">
</head>

<body>

  <!-- HEADER -->
  <header class="header">
    <div class="brand">
                    <img       src="{{      url_for('static',
filename='/images/blue.png')        }}"        width="150px"
height="150px"></div>
    </div>

    <nav class="nav">
      <a href="{{ url_for('home') }}" class="{% if active_page
== 'home' %}active{% endif %}">Home</a>
        <a  href="{{  url_for('features')  }}"  class="{%  if
active_page == 'features' %}active{% endif %}">Features</a>
      <a href="{{ url_for('demo') }}" class="{% if active_page
== 'demo' %}active{% endif %}">Demo</a>
        <a  href="{{  url_for('privacy')  }}"  class="{%  if
active_page == 'privacy' %}active{% endif %}">Privacy</a>
      <a href="{{ url_for('help') }}" class="{% if active_page
== 'help' %}active{% endif %}">Help</a>
```

```
        <a href="{{ url_for('feedback') }}" class="{% if
active_page == 'feedback' %}active{% endif %}">Feedback</a>
    </nav>

    <a href="{{ url_for('login') }}" class="login">Login</a>

  </header>
  <!-- LOGIN FORM -->
  <div class="login-container">
    <div class="login-card">
      <h2>Login to MiniGit</h2>
      <form method="POST" action="/login">
                    <input    type="text"    name="username"
placeholder="Username" required>
                  <input   type="password"   name="password"
placeholder="Password" required>
        <button type="submit">Login</button>
      </form>
         <p>Don't have an account? <a href="/register"
style="color: cyan; text-decoration: none;">Register</a></p>
    </div>
  </div>

</body>

</html>
```

create_repo.html:



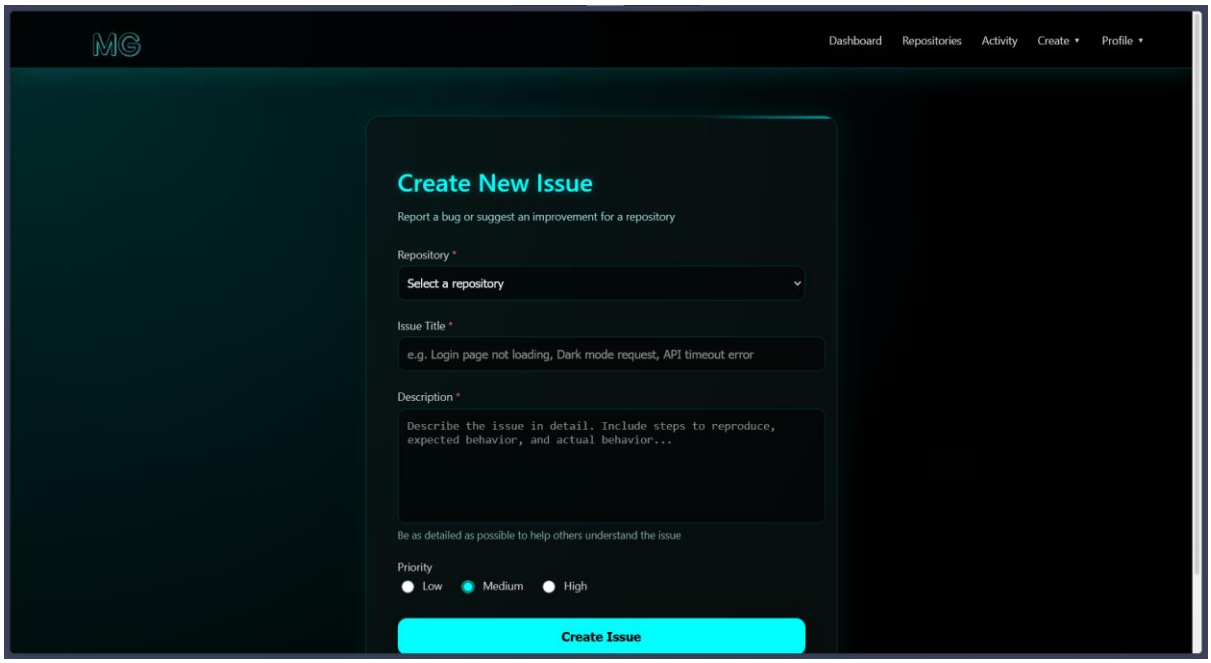
- **Source Code:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Create Repository | MiniGit</title>
    <link   rel="stylesheet"   href="{{   url_for('static',
filename='css/create_repo.css') }}">
</head>
<body>

<!-- ================ AUTH NAVBAR ================ -->
<header class="auth-header">
  <div class="auth-brand">
                <img        src="{{       url_for('static',
filename='/images/blue.png')        }}"        width="150px"
height="150px"></div>
  </div>

  <nav class="auth-nav">
      <a  href="/dashboard"  class="{%  if  active_page  ==
'dashboard' %}active{% endif %}">Dashboard</a>
     <a  href="/repositories"  class="{%  if  active_page  ==
'repositories' %}active{% endif %}">Repositories</a>
    <a href="/activity" class="{% if active_page == 'activity'
%}active{% endif %}">Activity</a>

    <!-- Create Dropdown -->
    <div class="dropdown">
      <span>Create ▾</span>
      <div class="dropdown-menu">
         <a href="/create-repo" class="{% if active_page ==
'create-repo' %}active{% endif %}">New Repository</a>
        <a href="/create-issue" class="{% if active_page ==
'create-issue' %}active{% endif %}">New Issue</a>
       <a href="/create-commit" class="{% if active_page ==
'create-commit' %}active{% endif %}">New Commit</a>
      </div>
    </div>

    <!-- Profile Dropdown -->
    <div class="dropdown profile-dropdown">
       <span  class="{%  if  active_page  in  ['profile',  'my-
repos', 'activity'] %}active{% endif %}">Profile ▾</span>
      <div class="dropdown-menu">
          <a  href="/profile"  class="{%  if  active_page  ==
'profile' %}active{% endif %}">View Profile</a>
        <a href="/repositories" class="{% if active_page ==
'my-repos' %}active{% endif %}">My Repositories</a>
         <a  href="/activity"  class="{%  if  active_page  ==
'activity' %}active{% endif %}">My Activity</a>
```

```
        <a href="/logout" class="logout">Logout</a>
      </div>
    </div>
  </nav>

</header>

<!-- ================ CREATE REPO SECTION ================
-->
<section class="create-container">
  <div class="repo-card">
    <h1>Create a New Repository</h1>
    <p class="subtitle">
      Set up a new MiniGit repository for your project
    </p>

  <form method="POST" action="/create-repo">
  <div class="form-group">
    <label>Repository Name</label>
    <input
      type="text"
      <!-- name="repo_name"  ADD THIS
      placeholder="e.g. mini-git-project"
      required
    >
  </div>

  <div class="form-group">
    <label>Description</label>
    <textarea
      name="description"  ADD THIS
      placeholder="Short description of your repository"
    ></textarea>
  </div>

  <div class="form-group">
    <label>Visibility</label>
    <div class="radio-group">
      <label>
        <input type="radio" name="visibility" value="public"
checked>  <!-- ADD value="public" -->
        Public
      </label>
      <label>
                  <input   type="radio"   name="visibility"
value="private">  <!-- ADD value="private" -->
        Private
      </label>
    </div>
  </div>
```

```
  <div class="form-group checkbox">
    <label>
              <input    type="checkbox"    name="init_readme"
value="yes">  <!-- ADD name="init_readme" -->
      Initialize repository with README
    </label>
  </div>

  <button type="submit" class="create-btn">
    Create Repository
  </button>
</form>
  </div>
</section>

</body>
</html>
```

create_issue:



- **Source Code:**

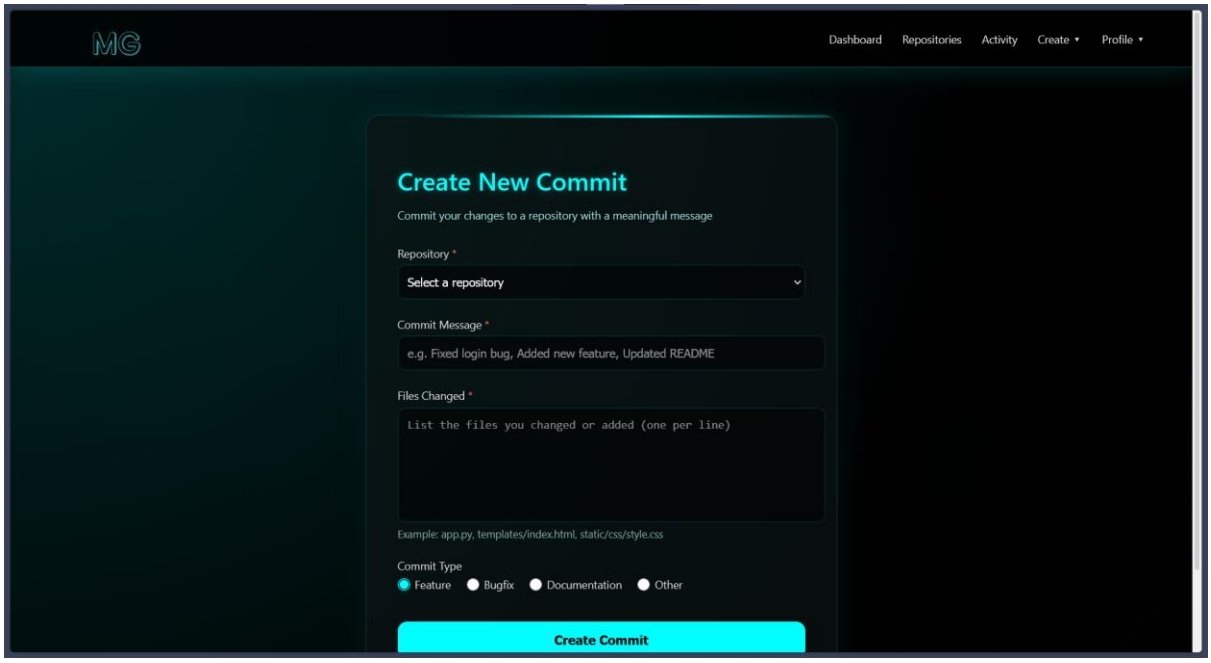
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Create Issue | MiniGit</title>
    <link    rel="stylesheet"    href="{{    url_for('static',
filename='css/create_issue.css') }}">
</head>
```

```
<body>

<!-- ================ AUTH NAVBAR ================ -->
<header class="auth-header">
  <div class="auth-brand">
                <img       src="{{       url_for('static',
filename='/images/blue.png')        }}"        width="150px"
height="150px">
  </div>

  <nav class="auth-nav">
      <a  href="/dashboard"  class="{%  if  active_page  ==
'dashboard' %}active{% endif %}">Dashboard</a>
     <a  href="/repositories"  class="{%  if  active_page  ==
'repositories' %}active{% endif %}">Repositories</a>
   <a href="/activity" class="{% if active_page == 'activity'
%}active{% endif %}">Activity</a>

    <!-- Create Dropdown -->
    <div class="dropdown">
      <span>Create ▾</span>
      <div class="dropdown-menu">
         <a href="/create-repo" class="{% if active_page ==
'create-repo' %}active{% endif %}">New Repository</a>
        <a href="/create-issue" class="{% if active_page ==
'create-issue' %}active{% endif %}">New Issue</a>
       <a href="/create-commit" class="{% if active_page ==
'create-commit' %}active{% endif %}">New Commit</a>
      </div>
    </div>

    <!-- Profile Dropdown -->
    <div class="dropdown profile-dropdown">
       <span  class="{%  if  active_page  in  ['profile',  'my-
repos', 'activity'] %}active{% endif %}">Profile ▾</span>
      <div class="dropdown-menu">
          <a  href="/profile"  class="{%  if  active_page  ==
'profile' %}active{% endif %}">View Profile</a>
        <a href="/repositories" class="{% if active_page ==
'my-repos' %}active{% endif %}">My Repositories</a>
          <a  href="/activity"  class="{%  if  active_page  ==
'activity' %}active{% endif %}">My Activity</a>
        <a href="/logout" class="logout">Logout</a>
      </div>
    </div>
  </nav>
</header>

<!-- ================ CREATE ISSUE SECTION ================
-->
```

```
<section class="issue-container">
  <div class="issue-card">
    <h1>Create New Issue</h1>
    <p class="subtitle">
      Report a bug or suggest an improvement for a repository
    </p>

    <form method="POST" action="/create-issue">

      <div class="form-group">
                  <label>Repository   <span   style="color:
#ff6b6b;">*</span></label>
        <select name="repo_id" required>
          <option value="">Select a repository</option>
          {% for repo in repositories %}
         <option value="{{ repo.repo_id }}">{{ repo.repo_name
}}</option>
          {% endfor %}
        </select>
      </div>

      <div class="form-group">
                <label>Issue   Title   <span   style="color:
#ff6b6b;">*</span></label>
        <input
          type="text"
          name="issue_title"
         placeholder="e.g. Login page not loading, Dark mode
request, API timeout error"
          required
        >
      </div>

      <div class="form-group">
                  <label>Description   <span   style="color:
#ff6b6b;">*</span></label>
        <textarea
          name="description"
          placeholder="Describe the issue in detail. Include
steps   to   reproduce,   expected   behavior,   and   actual
behavior..."
          required
        ></textarea>
        <small style="color: #88aaaa; display: block; margin-
top: 5px;">
         Be as detailed as possible to help others understand
the issue
        </small>
      </div>
```

```
        <div class="form-group">
          <label>Priority</label>
          <div class="radio-group">
            <label class="radio-label">
              <input type="radio" name="priority" value="low">
              <span>Low</span>
            </label>
            <label class="radio-label">
             <input type="radio" name="priority" value="medium"
checked>
              <span>Medium</span>
            </label>
            <label class="radio-label">
              <input type="radio" name="priority" value="high">
              <span>High</span>
            </label>
          </div>
        </div>

        <button type="submit" class="create-btn">
          Create Issue
        </button>

      </form>
    </div>
  </section>

  </body>
  </html>
```

create_commit:



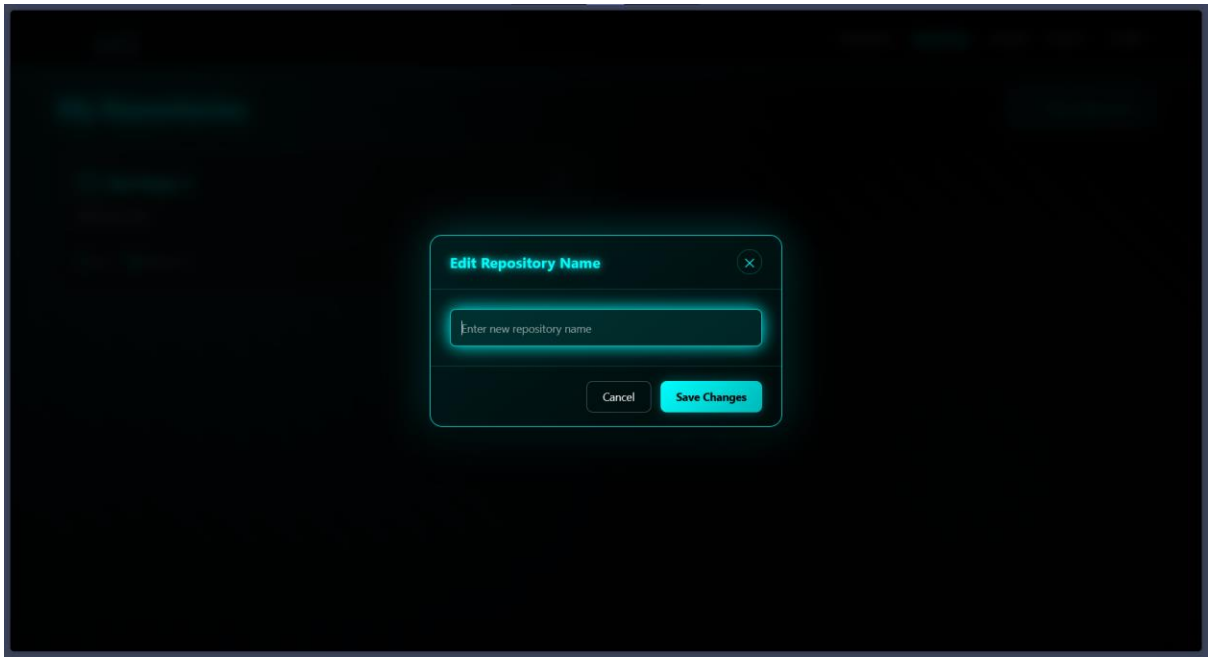
- **Source Code:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Create Commit | MiniGit</title>
    <link   rel="stylesheet"   href="{{   url_for('static',
filename='css/create_commit.css') }}">
</head>
<body>

<!-- ================ AUTH NAVBAR ================ -->
<header class="auth-header">
  <div class="auth-brand">
                <img       src="{{       url_for('static',
filename='/images/blue.png')       }}"       width="150px"
height="150px">
  </div>

  <nav class="auth-nav">
      <a  href="/dashboard"  class="{%  if  active_page  ==
'dashboard' %}active{% endif %}">Dashboard</a>
     <a  href="/repositories"  class="{%  if  active_page  ==
'repositories' %}active{% endif %}">Repositories</a>
    <a href="/activity" class="{% if active_page == 'activity'
%}active{% endif %}">Activity</a>

    <!-- Create Dropdown -->
    <div class="dropdown">
      <span>Create ▾</span>
      <div class="dropdown-menu">
         <a href="/create-repo" class="{% if active_page ==
'create-repo' %}active{% endif %}">New Repository</a>
        <a href="/create-issue" class="{% if active_page ==
'create-issue' %}active{% endif %}">New Issue</a>
       <a href="/create-commit" class="{% if active_page ==
'create-commit' %}active{% endif %}">New Commit</a>
      </div>
    </div>

    <!-- Profile Dropdown -->
    <div class="dropdown profile-dropdown">
       <span  class="{%  if  active_page  in  ['profile',  'my-
repos', 'activity'] %}active{% endif %}">Profile ▾</span>
      <div class="dropdown-menu">
          <a  href="/profile"  class="{%  if  active_page  ==
'profile' %}active{% endif %}">View Profile</a>
        <a href="/repositories" class="{% if active_page ==
'my-repos' %}active{% endif %}">My Repositories</a>
```

```
          <a href="/activity" class="{% if active_page ==
'activity' %}active{% endif %}">My Activity</a>
          <a href="/logout" class="logout">Logout</a>
        </div>
      </div>
    </nav>
</header>

<!--     ================     CREATE     COMMIT     SECTION
================ -->
<section class="commit-container">
  <div class="commit-card">
    <h1>Create New Commit</h1>
    <p class="subtitle">
      Commit your changes to a repository with a meaningful
message
    </p>

    <form method="POST" action="/create-commit">

      <div class="form-group">
                  <label>Repository   <span   style="color:
#ff6b6b;">*</span></label>
        <select name="repo_id" required>
          <option value="">Select a repository</option>
          {% for repo in repositories %}
         <option value="{{ repo.repo_id }}">{{ repo.repo_name
}}</option>
          {% endfor %}
        </select>
      </div>

      <div class="form-group">
              <label>Commit   Message   <span   style="color:
#ff6b6b;">*</span></label>
        <input
          type="text"
          name="commit_message"
             placeholder="e.g.  Fixed  login  bug,  Added  new
feature, Updated README"
          required
        >
      </div>

      <div class="form-group">
               <label>Files   Changed   <span   style="color:
#ff6b6b;">*</span></label>
        <textarea
          name="files_changed"
```

```
            placeholder="List the files you changed or added
(one per line)"
            required
          ></textarea>
          <small style="color: #88aaaa; display: block; margin-
top:     5px;">Example:     app.py,     templates/index.html,
static/css/style.css</small>
        </div>

        <div class="form-group">
          <label>Commit Type</label>
          <div class="radio-group">
            <label class="radio-label">
                      <input  type="radio"  name="commit_type"
value="feature" checked>
              <span>Feature </span>
            </label>
            <label class="radio-label">
                      <input  type="radio"  name="commit_type"
value="bugfix">
              <span>Bugfix </span>
            </label>
            <label class="radio-label">
                      <input  type="radio"  name="commit_type"
value="documentation">
              <span>Documentation </span>
            </label>
            <label class="radio-label">
                      <input  type="radio"  name="commit_type"
value="other">
              <span>Other </span>
            </label>
          </div>
        </div>

        <button type="submit" class="create-btn">
          Create Commit
        </button>

      </form>
    </div>
  </section>

  </body>
  </html>
```

repositories.html:



- **Source Code:**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Repositories | MiniGit</title>
    <link
      rel="stylesheet"
                              href="{{        url_for('static',
filename='css/repositories.css') }}"
    />
  </head>
  <body>
    <!-- ================ AUTH NAVBAR ================ -->
    <header class="auth-header">
      <div class="auth-brand">
                          <img     src="{{     url_for('static',
filename='/images/blue.png')        }}"        width="150px"
height="150px">
      </div>

      <nav class="auth-nav">
          <a  href="/dashboard"  class="{%  if  active_page  ==
'dashboard' %}active{% endif %}">Dashboard</a>
        <a href="/repositories" class="{% if active_page ==
'repositories' %}active{% endif %}">Repositories</a>
          <a  href="/activity"  class="{%  if  active_page  ==
'activity' %}active{% endif %}">Activity</a>

        <!-- Create Dropdown -->
        <div class="dropdown">
```

```
            <span>Create ▾</span>
            <div class="dropdown-menu">
               <a href="/create-repo" class="{% if active_page
== 'create-repo' %}active{% endif %}">New Repository</a>
              <a href="/create-issue" class="{% if active_page
== 'create-issue' %}active{% endif %}">New Issue</a>
             <a href="/create-commit" class="{% if active_page
== 'create-commit' %}active{% endif %}">New Commit</a>
            </div>
          </div>
 <script>
  // Toggle dropdown menu
  function toggleDropdown(button) {
    const dropdown = button.nextElementSibling;
    dropdown.classList.toggle('show');

    // Close other dropdowns
                              document.querySelectorAll('.repo-
menu.show').forEach(menu => {
      if (menu !== dropdown) menu.classList.remove('show');
    });
  }

  // Close dropdowns when clicking outside
  document.addEventListener('click', function(event) {
    if (!event.target.closest('.repo-dropdown')) {
                              document.querySelectorAll('.repo-
menu.show').forEach(menu => {
        menu.classList.remove('show');
      });
    }
  });

  // Edit repository
  let currentEditRepoId = null;

  function editRepo(repoId) {
    currentEditRepoId = repoId;
    const modal = document.getElementById('editModal');

    // Get current repo name
     const repoCard = document.querySelector(`.repository-
card[data-repo-id="${repoId}"]`);
        const currentName = repoCard.querySelector('h3
a').textContent;

           document.getElementById('newRepoName').value =
currentName;
    modal.style.display = 'flex';
```

```
    // Close any open dropdowns
                              document.querySelectorAll('.repo-
menu.show').forEach(menu => {
      menu.classList.remove('show');
    });
  }

  // Save repository name
  function saveRepoName() {
                           const            newName            =
document.getElementById('newRepoName').value.trim();

    if (!newName) {
      alert('Repository name cannot be empty');
      return;
    }

    if (newName.length < 3) {
      alert('Repository name must be at least 3 characters');
      return;
    }

    // Show loading state
       const   saveBtn   =   document.querySelector('.modal-
btn.save');
    const originalText = saveBtn.textContent;
    saveBtn.textContent = 'Saving...';
    saveBtn.disabled = true;

    // Simulate successful save for demo
    setTimeout(() => {
      // Update the repo name in the card
       const repoCard = document.querySelector(`.repository-
card[data-repo-id="${currentEditRepoId}"]`);
      const repoLink = repoCard.querySelector('h3 a');
      repoLink.textContent = newName;
      repoCard.dataset.repoName = newName;

      closeModal();
      alert('Repository name updated successfully!');

      saveBtn.textContent = originalText;
      saveBtn.disabled = false;
    }, 1000);
  }

  // Toggle visibility
  function toggleVisibility(repoId) {
     const  repoCard  =  document.querySelector(`.repository-
card[data-repo-id="${repoId}"]`);
```

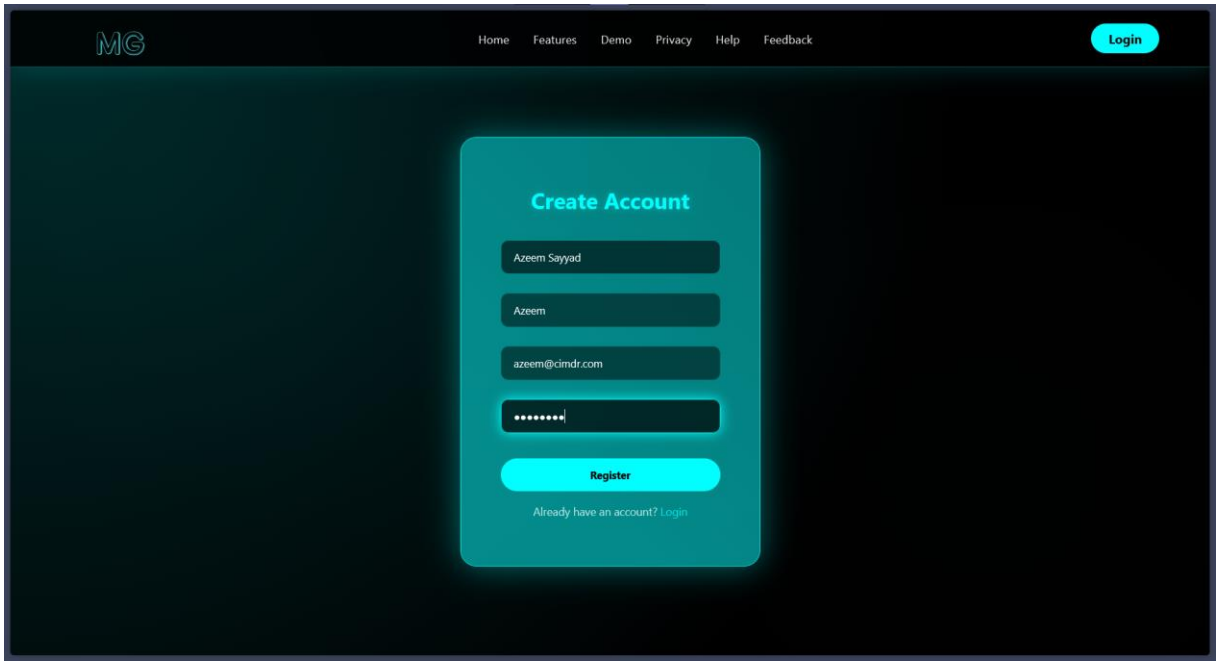
```
    const currentVisibility = repoCard.dataset.visibility;
     const newVisibility = currentVisibility === 'public' ?
'private' : 'public';

    // Show loading state
                      const          visibilityLink          =
repoCard.querySelector('.visibility-text');
    const originalText = visibilityLink.textContent;
    visibilityLink.textContent = 'Updating...';

    // Simulate successful toggle for demo
    setTimeout(() => {
      // Update visibility in card
      repoCard.dataset.visibility = newVisibility;

      // Update badge
        const  badge  =  repoCard.querySelector('.visibility-
badge');
      badge.className = `visibility-badge ${newVisibility}`;
      badge.textContent = newVisibility;

      // Update dropdown option text
       const  newText  =  newVisibility  ===  'public'  ?  'Make
Private' : 'Make Public';
      visibilityLink.textContent = newText;

      alert(`Repository is now ${newVisibility}`);

      // Close dropdown
                             document.querySelectorAll('.repo-
menu.show').forEach(menu => {
        menu.classList.remove('show');
      });
    }, 500);
  }

  // Delete repository
  // Delete repository
function deleteRepo(repoId) {
     if (confirm('⚠ Are you sure you want to delete this
repository?\n\nThis action cannot be undone. All commits and
issues will be permanently deleted.')) {
       const repoCard = document.querySelector(`.repository-
card[data-repo-id="${repoId}"]`);

        // Show deleting state
        repoCard.style.opacity = '0.5';
        repoCard.style.pointerEvents = 'none';

        fetch(`/api/repo/${repoId}/delete`, {
```

```
            method: 'POST',
            headers: { 'Content-Type': 'application/json' }
        })
        .then(response => response.json())
        .then(data => {
            if (data.success) {
                // Remove the card with animation
                repoCard.style.transition = 'all 0.3s ease';
                repoCard.style.transform = 'scale(0.8)';
                repoCard.style.opacity = '0';

                setTimeout(() => {
                    repoCard.remove();

                    // Check if no repos left
                                    const remainingCards =
document.querySelectorAll('.repository-card');
                    if (remainingCards.length === 0) {
                        location.reload(); // Reload to show
empty state
                    } else {
                        showNotification('Repository deleted
successfully', 'success');
                    }
                }, 300);
            } else {
                repoCard.style.opacity = '1';
                repoCard.style.pointerEvents = 'auto';
                  showNotification(data.error || 'Could not
delete repository', 'error');
            }
        })
        .catch(error => {
            console.error('Error:', error);
            repoCard.style.opacity = '1';
            repoCard.style.pointerEvents = 'auto';
             showNotification('Failed to delete repository',
'error');
        });

        // Close dropdown
                            document.querySelectorAll('.repo-
menu.show').forEach(menu => {
            menu.classList.remove('show');
        });
    }
}

  // Close modal
  function closeModal() {
```

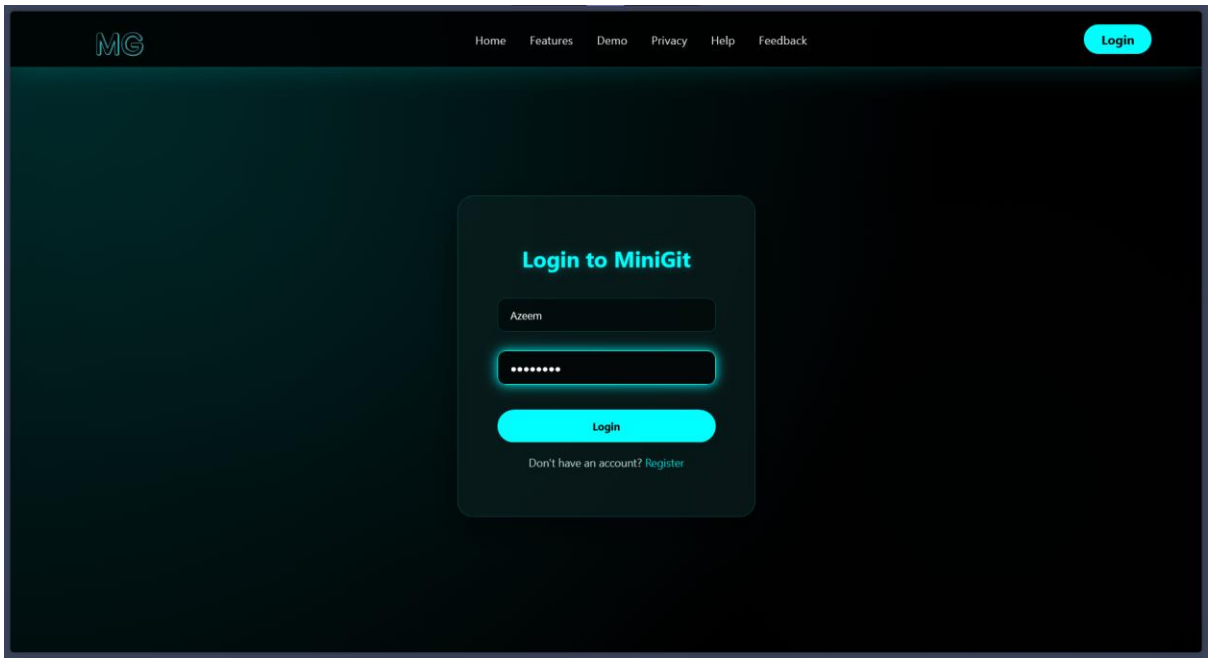
```
        document.getElementById('editModal').style.display  =
'none';
    document.getElementById('newRepoName').value = '';
    currentEditRepoId = null;
  }

  // Close modal when clicking outside
  window.onclick = function(event) {
    const modal = document.getElementById('editModal');
    if (event.target === modal) {
      closeModal();
    }
  }

  // Handle Enter key in modal
  document.addEventListener('keydown', function(event) {
            if     (event.key     ===     'Escape'     &&
document.getElementById('editModal').style.display      ===
'flex') {
      closeModal();
    }
             if     (event.key     ===     'Enter'     &&
document.getElementById('editModal').style.display      ===
'flex') {
      saveRepoName();
    }
  });
</script>
  </body>
</html>
```

- **With data:**

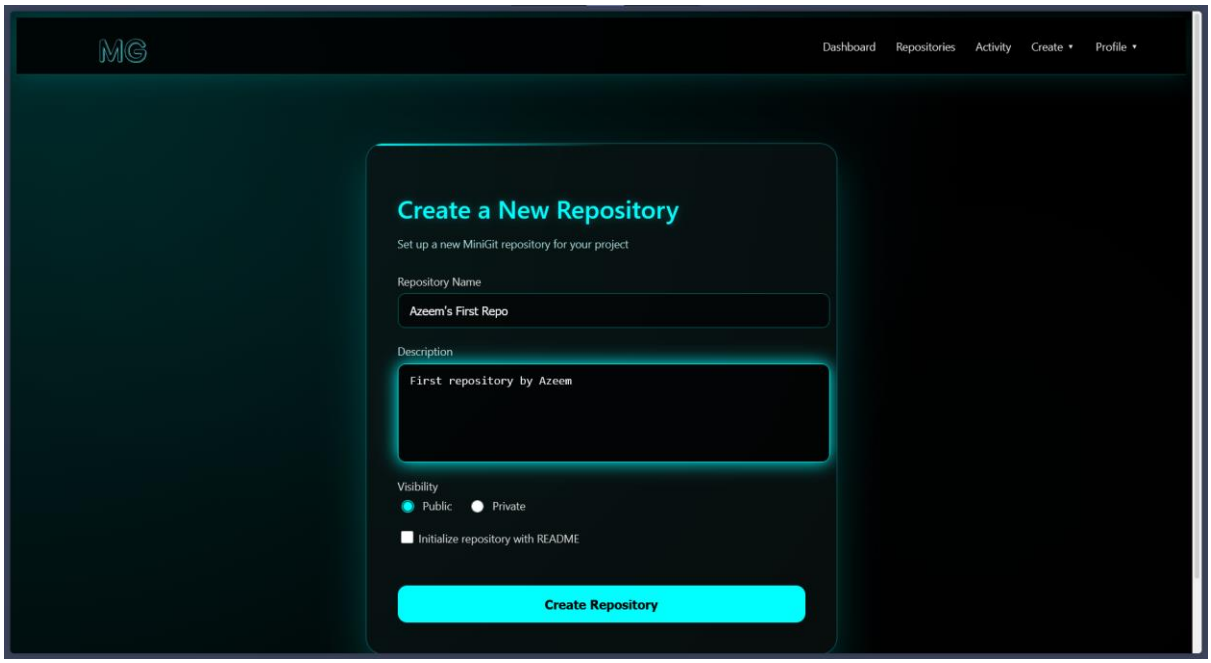
register.html:


MG
Home
Features
Demo
Privacy
Help
Feedback
Login
Create Account
Azeem Sayyad
Azeem
azeem@cimdr.com
Register
Already have an account? Login

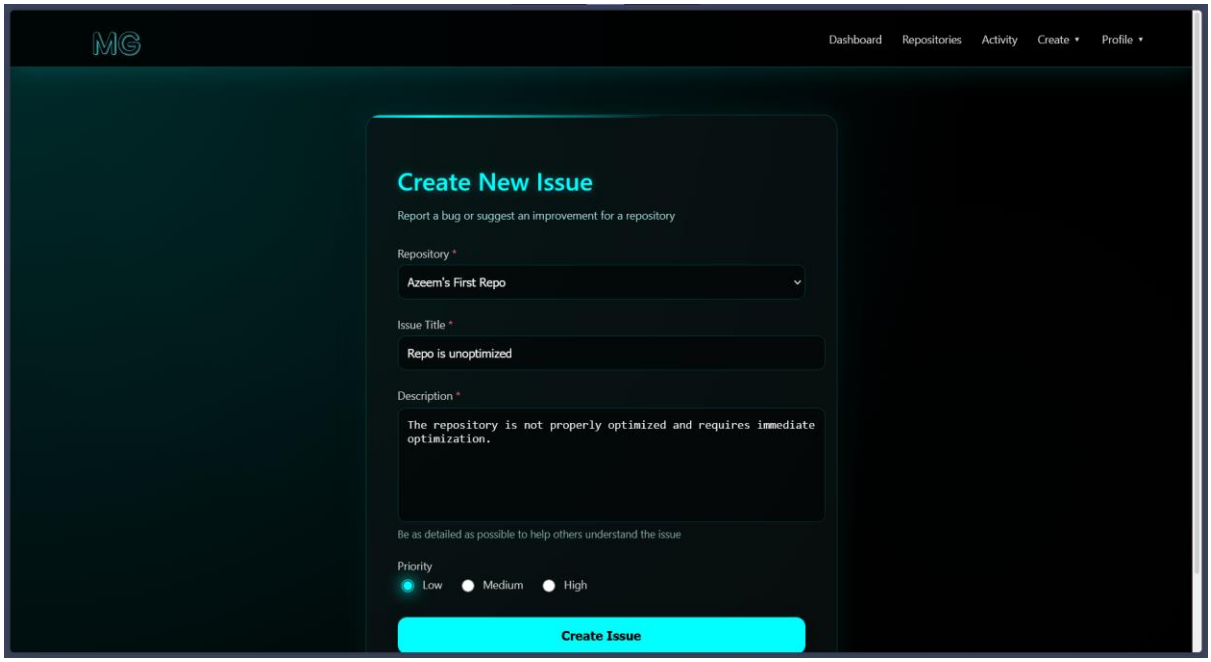
login.html:


MG
Home
Features
Demo
Privacy
Help
Feedback
Login
Login to MiniGit
Azeem
Login
Don't have an account? Register

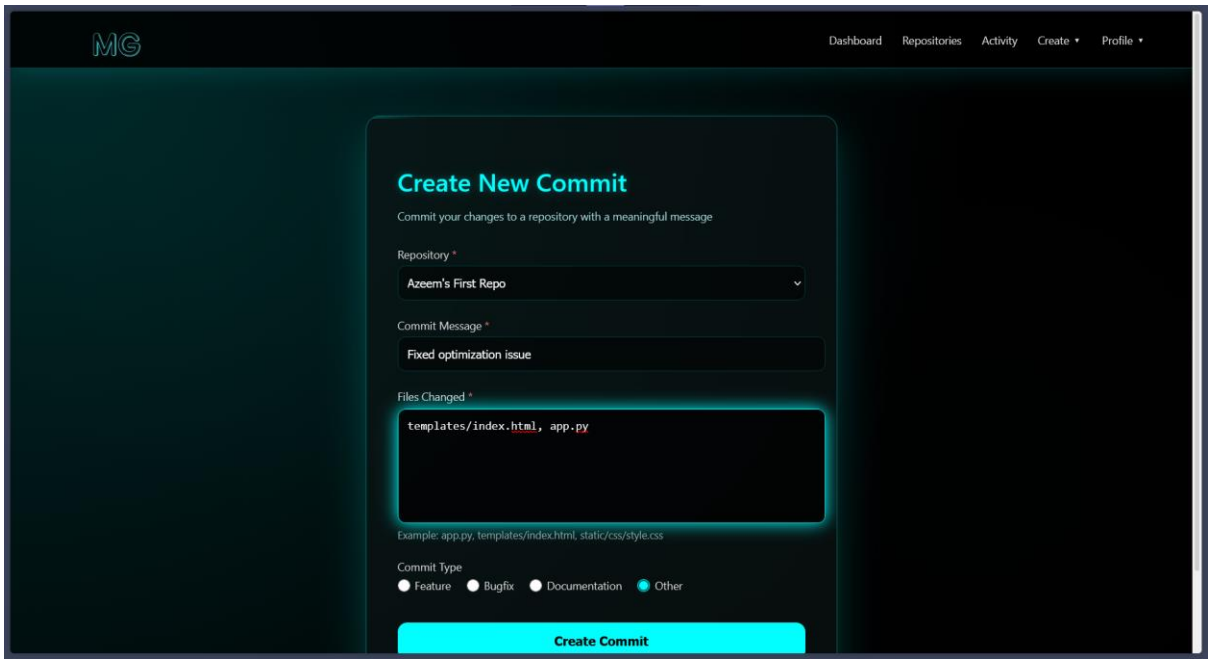
create_repo.html:



create_issue:



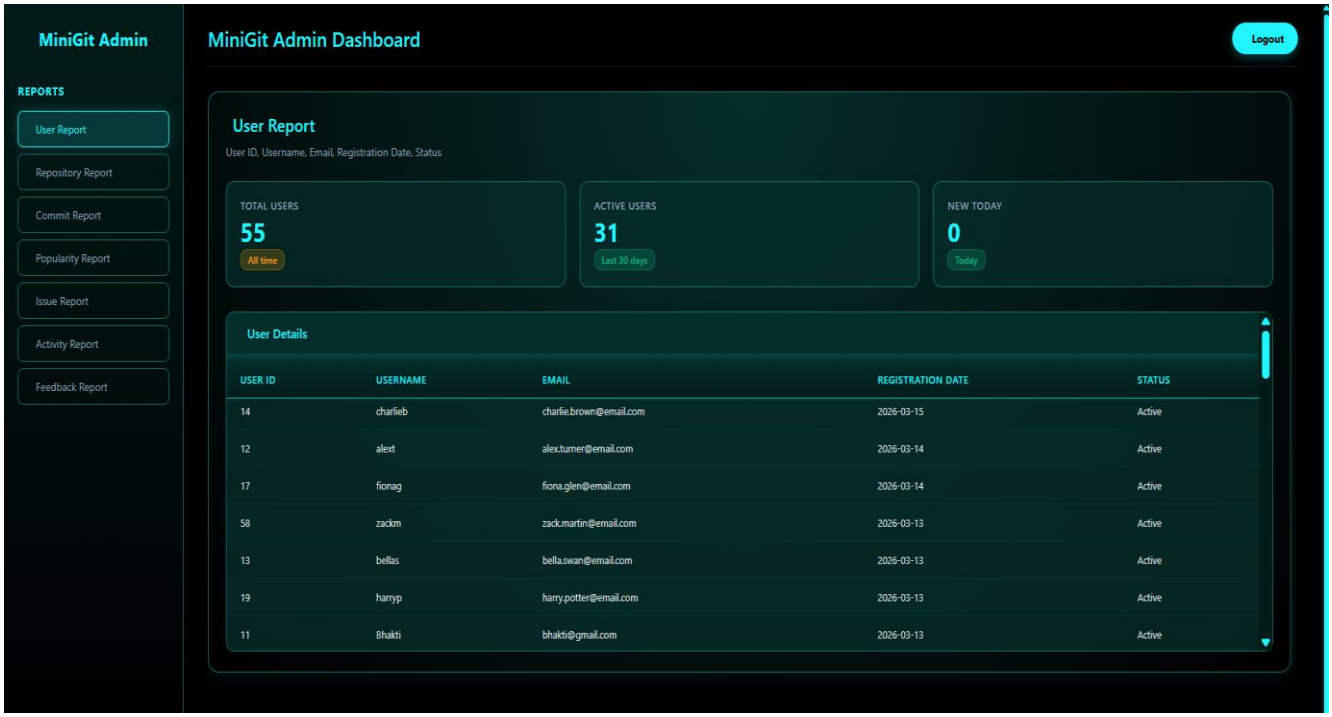
create_commit:



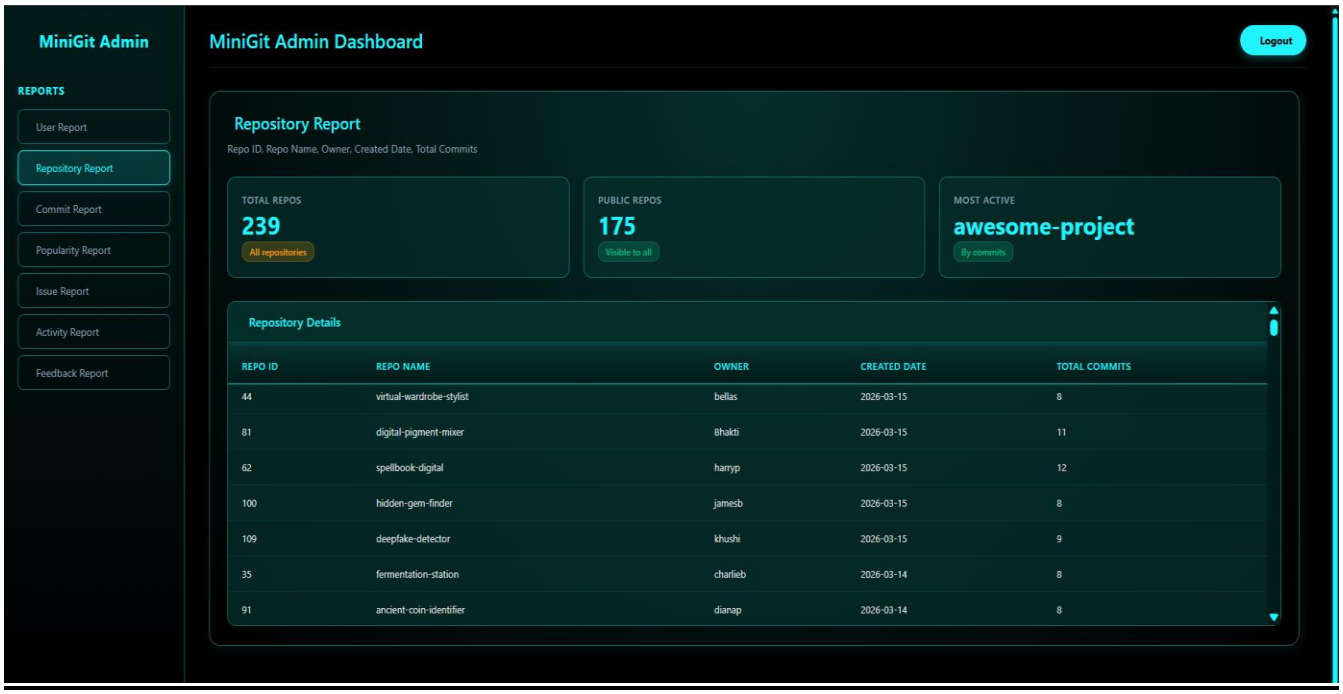
repositories.html:

4.3 Output Design:

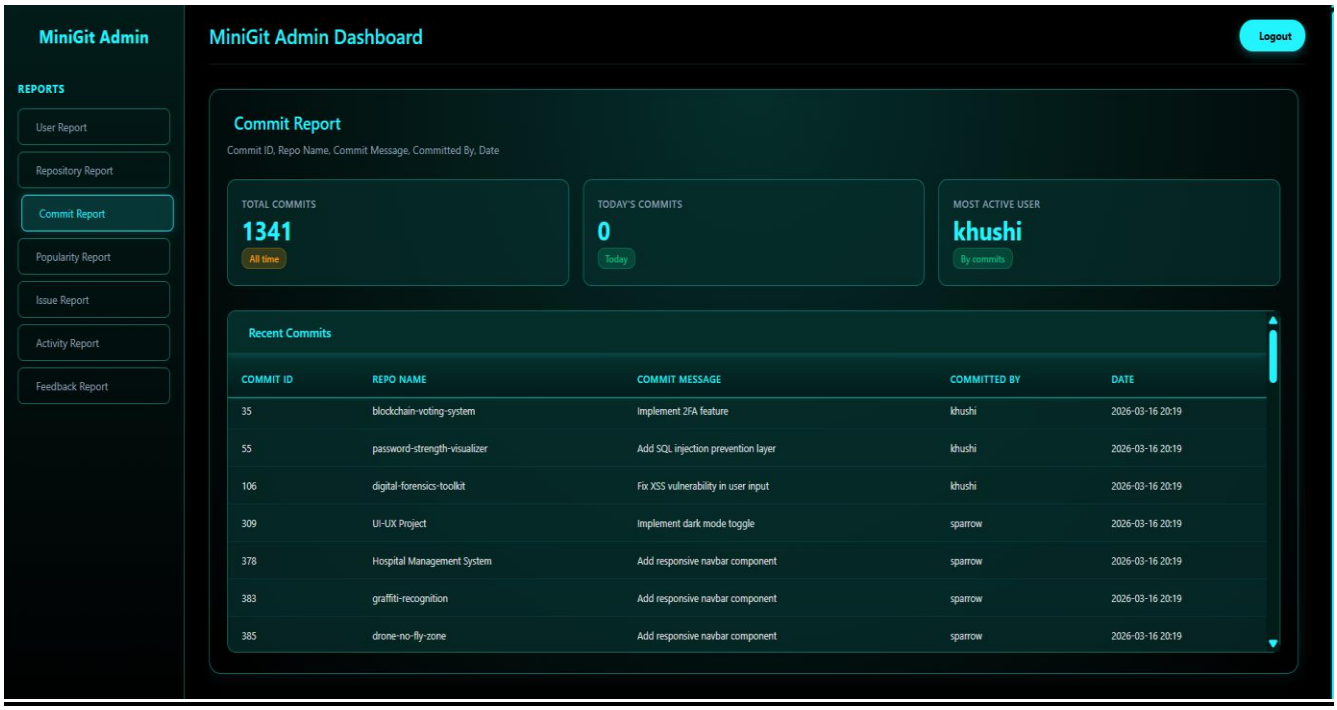
User Report:



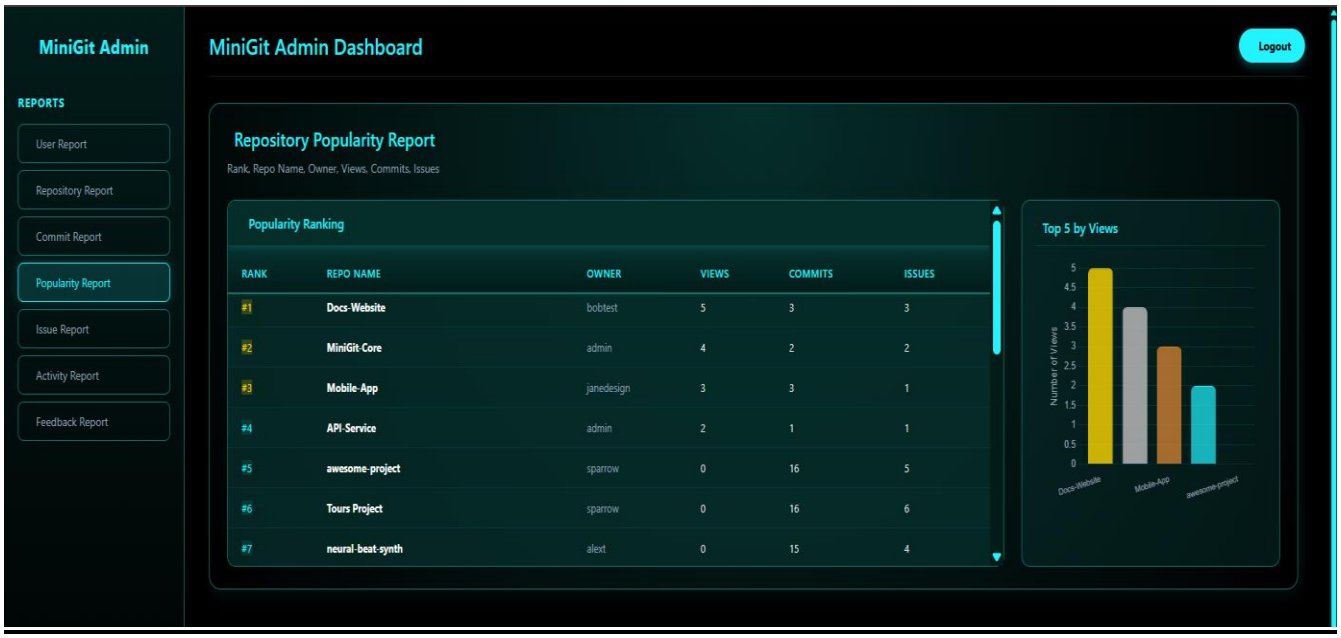
Repository Report:



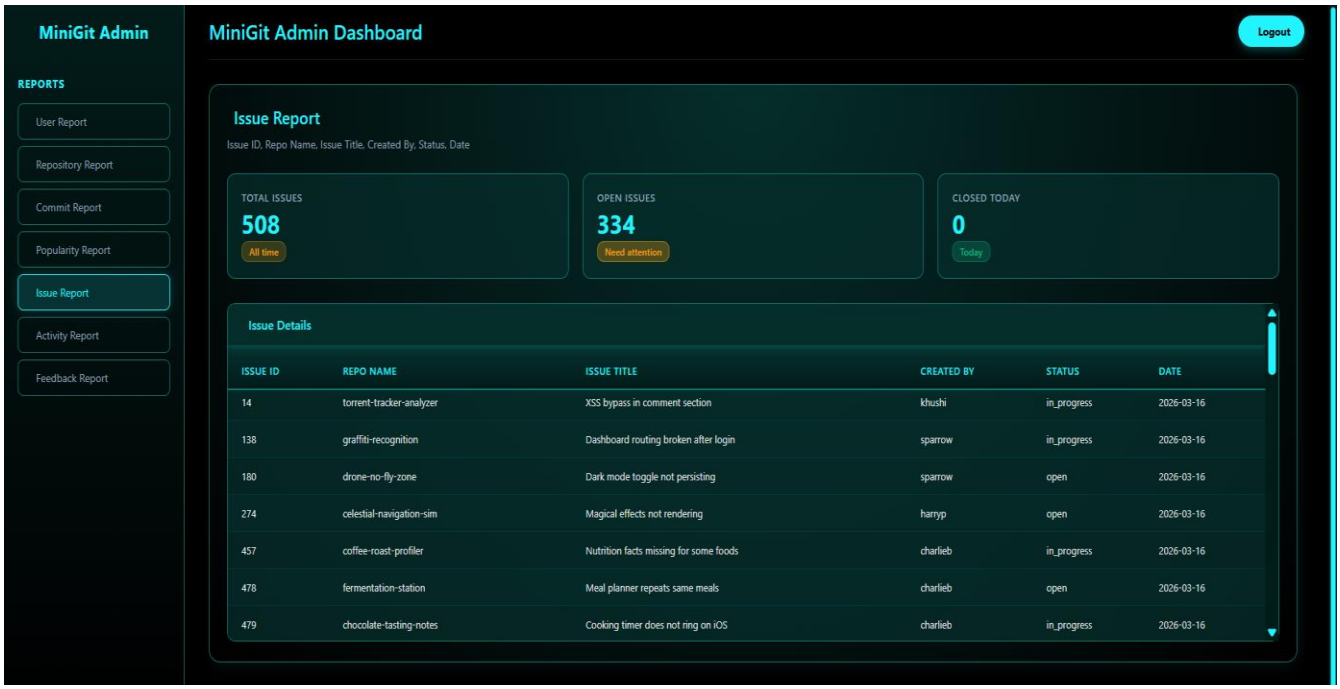
Commits Report:



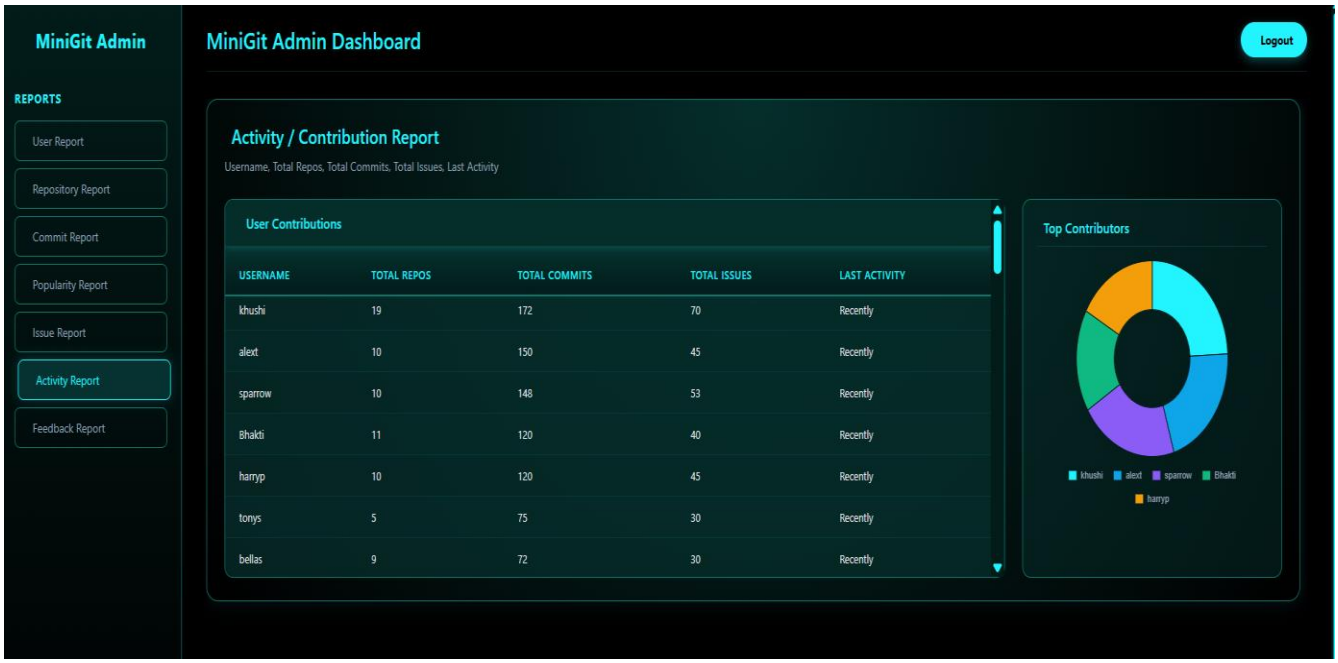
Popularity Report:



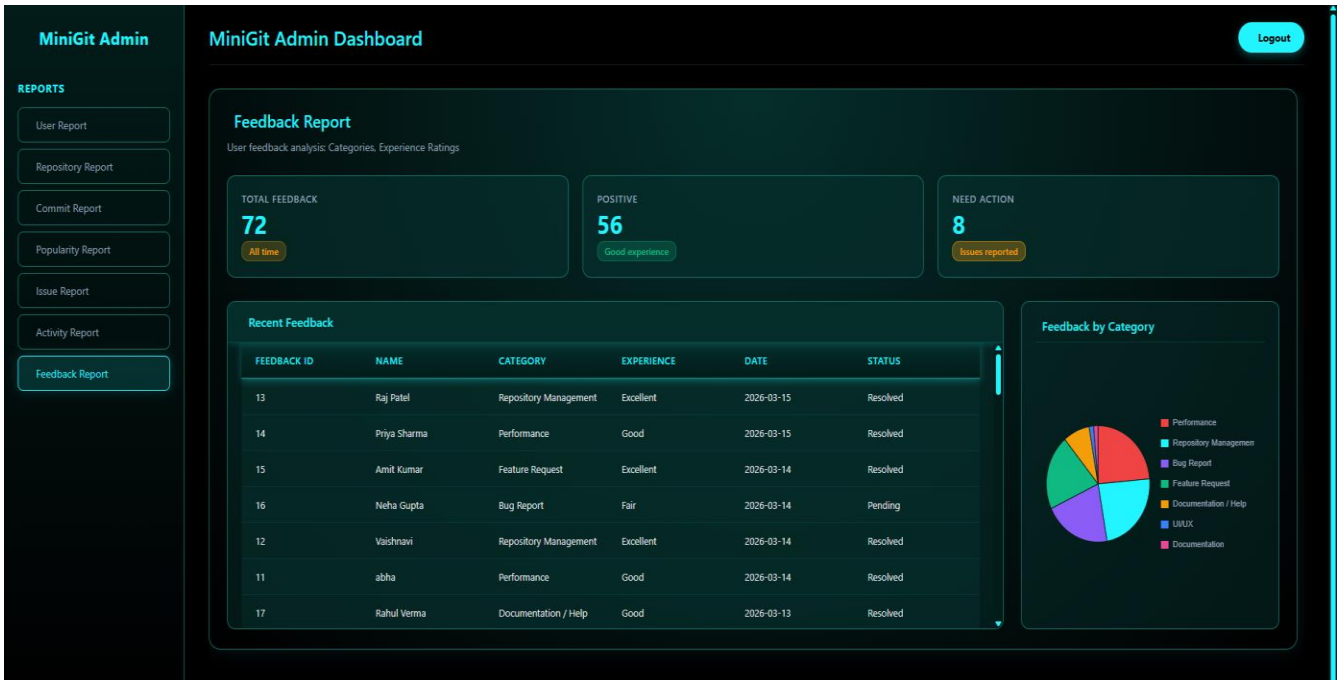
Issue Report:



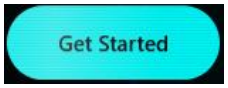
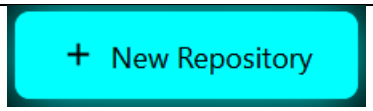
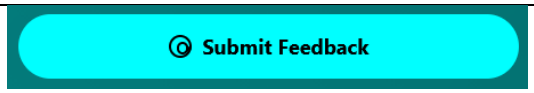
Activity Report:



Feedback Report:



5.1 User Manual:

Sr. No.	Tab/Button	Description
1)		Allows user/admin login.
2)	 	Allows user to register.
3)	 	Used to create repository.
4)		Used to create issue.
5)		Used to create commit.
6)		Used to edit and save repo's name.
7)		Used to submit feedback.

6.1 Conclusion:

In conclusion, MiniGit is a web-based version control management system developed using Flask and MySQL, designed to replicate the core functionalities of a modern source code hosting platform. The system enables users to create and manage repositories, track commits, raise and monitor issues, and engage with the platform through features such as repository starring and activity logging. By leveraging Flask as the backend framework and MySQL as the relational database, the project demonstrates the practical application of web development principles and structured data management in building a functional, multi-user software collaboration tool. MiniGit serves as a foundational implementation that reflects an understanding of user authentication, relational database design, and the organization of interconnected data entities in a real-world context.

6.2 Further Enhancements:

- **Branch and merge management —** The system could be extended to support the creation and merging of branches within repositories, allowing for a more complete and realistic version control workflow.
- **Pull request system —** A pull request mechanism could be introduced, enabling users to propose, review, and discuss code changes before they are merged into the main codebase.
- **Collaborative access control —** Role-based repository access could be implemented, allowing repository owners to add collaborators with varying levels of permissions such as read, write, or admin access.
- **Anonymous feedback linkage —** The current feedback module operates independently of the user system; linking it to authenticated users would enable more meaningful tracking and follow-up on submitted feedback.
- **Notification system —** A real-time or email-based notification system could be added to alert users of relevant activity such as new issues, commits, or stars on their repositories.
- **Search and filtering —** Enhanced search functionality across repositories, commits, and issues would significantly improve usability as the platform scales to accommodate a larger number of users and projects.

7.1 Bibliography:

Books Referred:

- **Flask Web Development — Miguel Grinberg**
- **Database Management Systems — Ramakrishnan & Gehrke**
- **Database System Concepts — Silberschatz, Korth & Sudarshan**

Website Referred:

- **https://www.w3schools.com/**
- **https://www.google.com/**
- **https://www.youtube.com/**